Chapter 2:
Operating-System Structures

Chapter 2: Operating-System Structures

- Operating System Services
- User Operating System Interface
- System Calls
- Types of System Calls
- System Programs
- Operating System Design and Implementation
- Operating System Structure
- Virtual Machines
- Operating System Debugging
- Operating System Generation
- System Boot

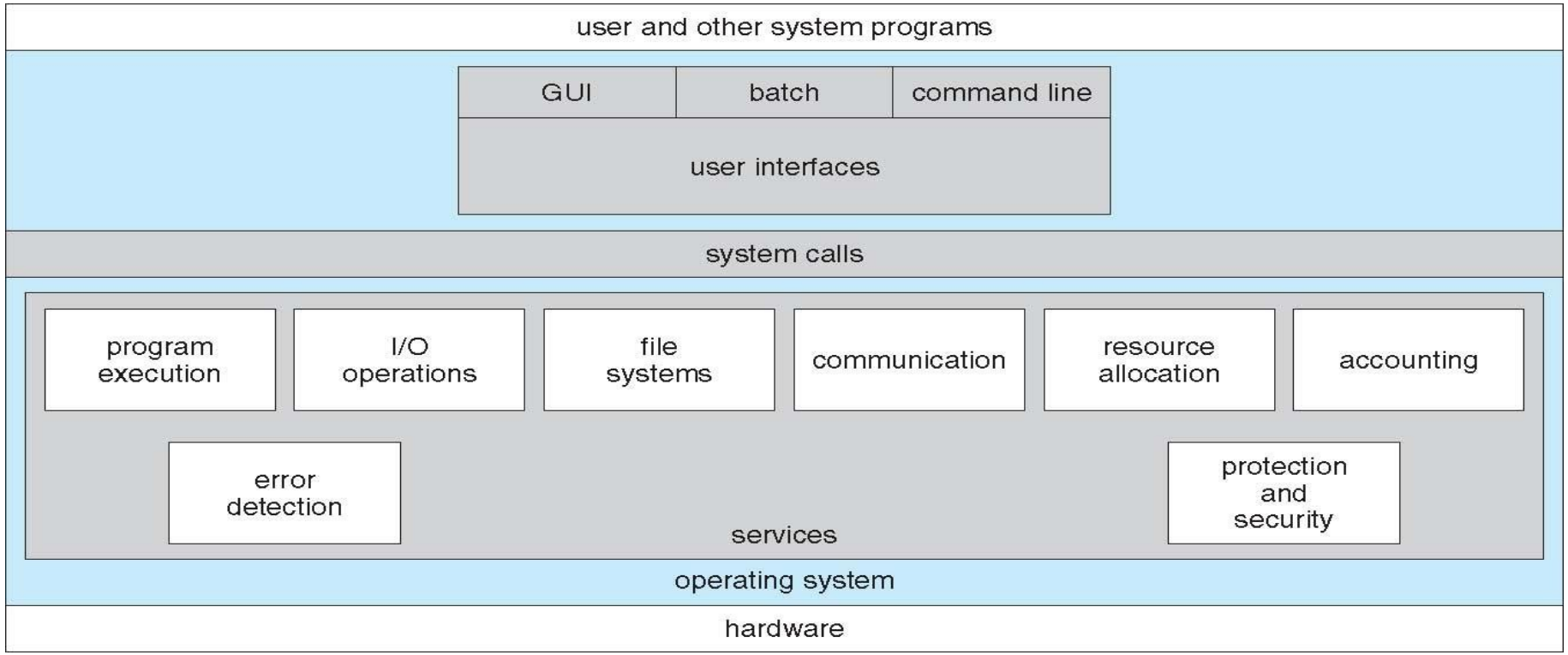
Objectives

- To describe the services an operating system provides to users, processes, and other systems
- To discuss the various ways of structuring an operating system
- To explain how operating systems are installed and customized and how they boot

Operating System Services

- One set of operating-system services provides functions that are helpful to the user:
  - **User interface** - Almost all operating systems have a user interface (UI)
    - Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **Batch Interface.**
  - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
  - **File-system manipulation** - The file system is of particular interest. Obviously, programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.

A View of Operating System Services



Operating System Services (Cont.)

- One set of operating-system services provides functions that are helpful to the user (cont.):
  - Communications – Processes may exchange information, on the same computer or between computers over a network
    - Communications may be via shared memory or through message passing (packets moved by the OS)
  - Error detection – OS needs to be constantly aware of possible errors
    - May occur in the CPU and memory hardware, in I/O devices, in user program
    - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
    - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

Operating System Services (Cont.)

- **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
  - Many types of resources - Some (such as CPU cycles, main memory, and file storage) may have special allocation code, others (such as I/O devices) may have general request and release code
- **Accounting** - To keep track of which users use how much and what kinds of computer resources
- **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
  - **Protection** involves ensuring that all access to system resources is controlled
  - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts
  - If a system is to be protected and secure, precautions must be instituted throughout it. A chain is only as strong as its weakest link.

System Calls

- ***System Calls*** *provide programming interface for user processes to request the services from the system made available by the OS.*

- **System calls** allow user-level processes to request services of the **operating system**
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Program Interface (API)** rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)
- Why use APIs rather than system calls?

System Calls and API (Def.)

- *A System Call is an explicit request to the kernel made via a software interrupt by OS from user processes.*

- *Application Programming Interface (API) is a set of functions, objects, protocols or data structures for the support of application development for developers/programmers.*

API (Def.)

- *Operating Systems provide a library (or high-level Application Program Interface or API) that sits between normal programs and the rest of the operating system.*

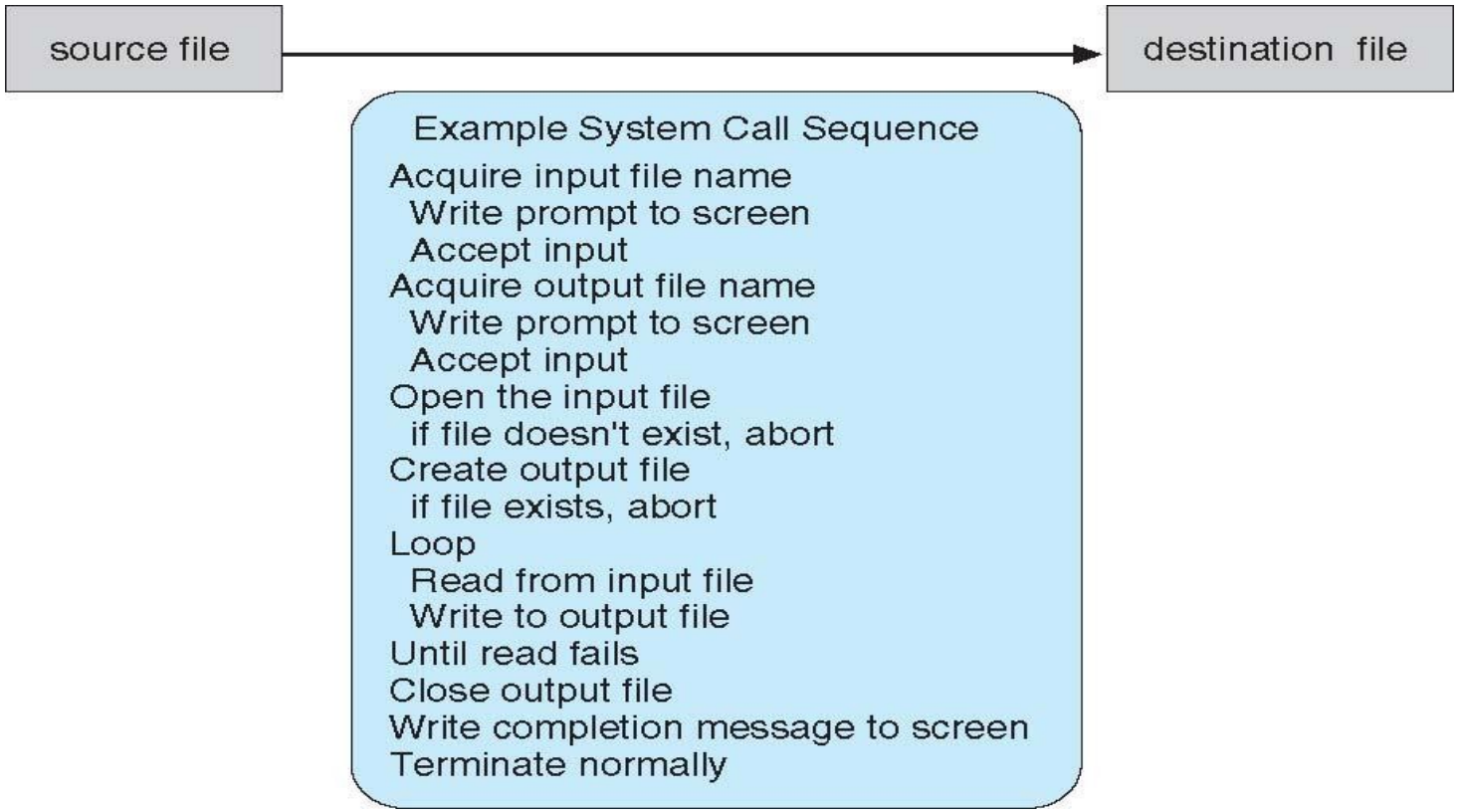
- *This library handles the low-level details of passing information to the kernel and switching to supervisor mode,*

API (Def.) Cont.

- *It(API) is actually a kind of function definition which specifies how to make available of a specific service of the system/OS.*
- *The API's are available from library or from Operating system itself.*
- *API is used in high level programming to invoke system calls in low level architecture.*

Example of System Calls

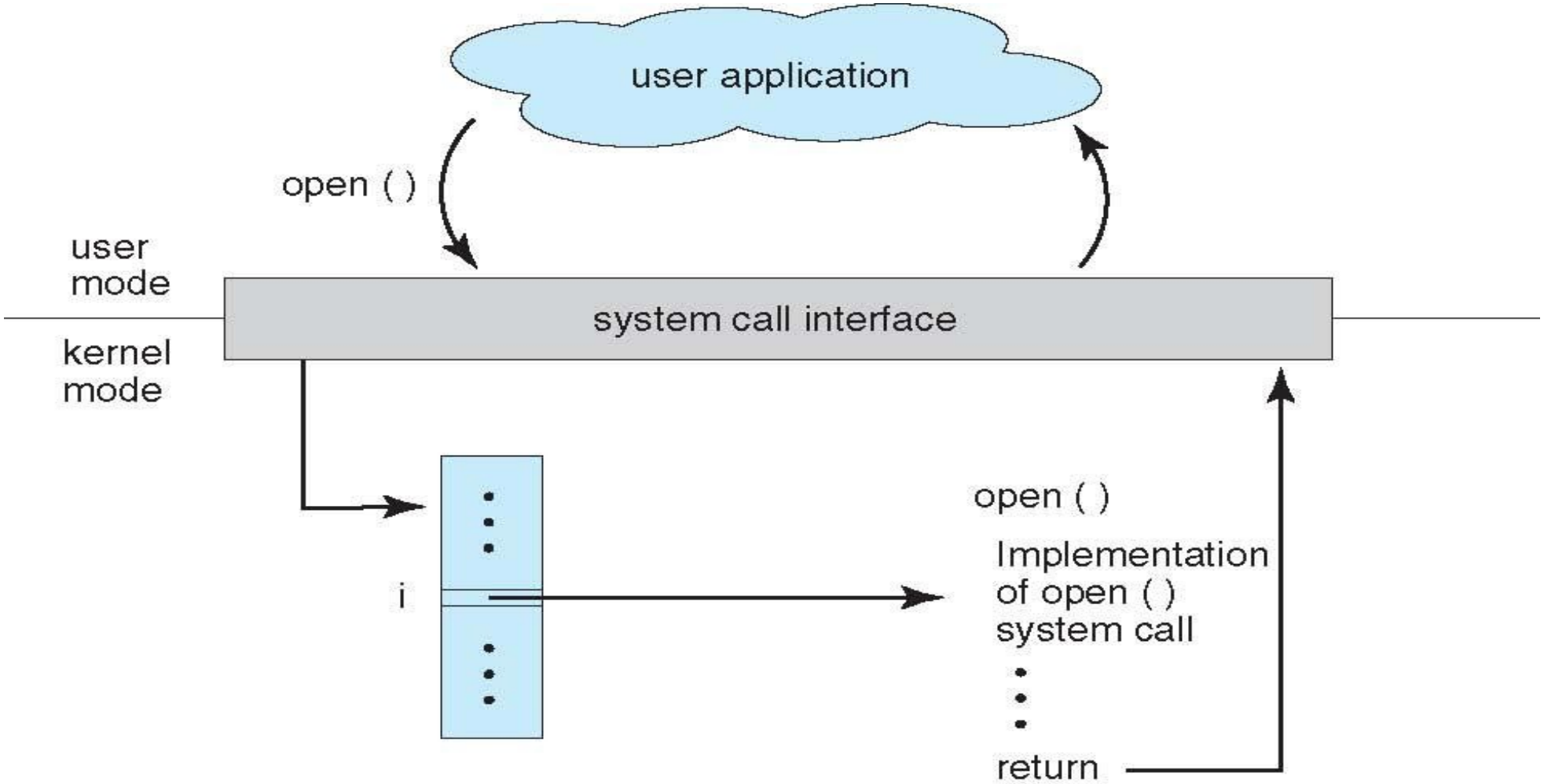
- System call sequence to copy the contents of one file to another file



System Call Implementation

- Typically, a number associated with each system call
  - System-call interface maintains a table indexed according to these numbers
- The system call interface invokes intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)

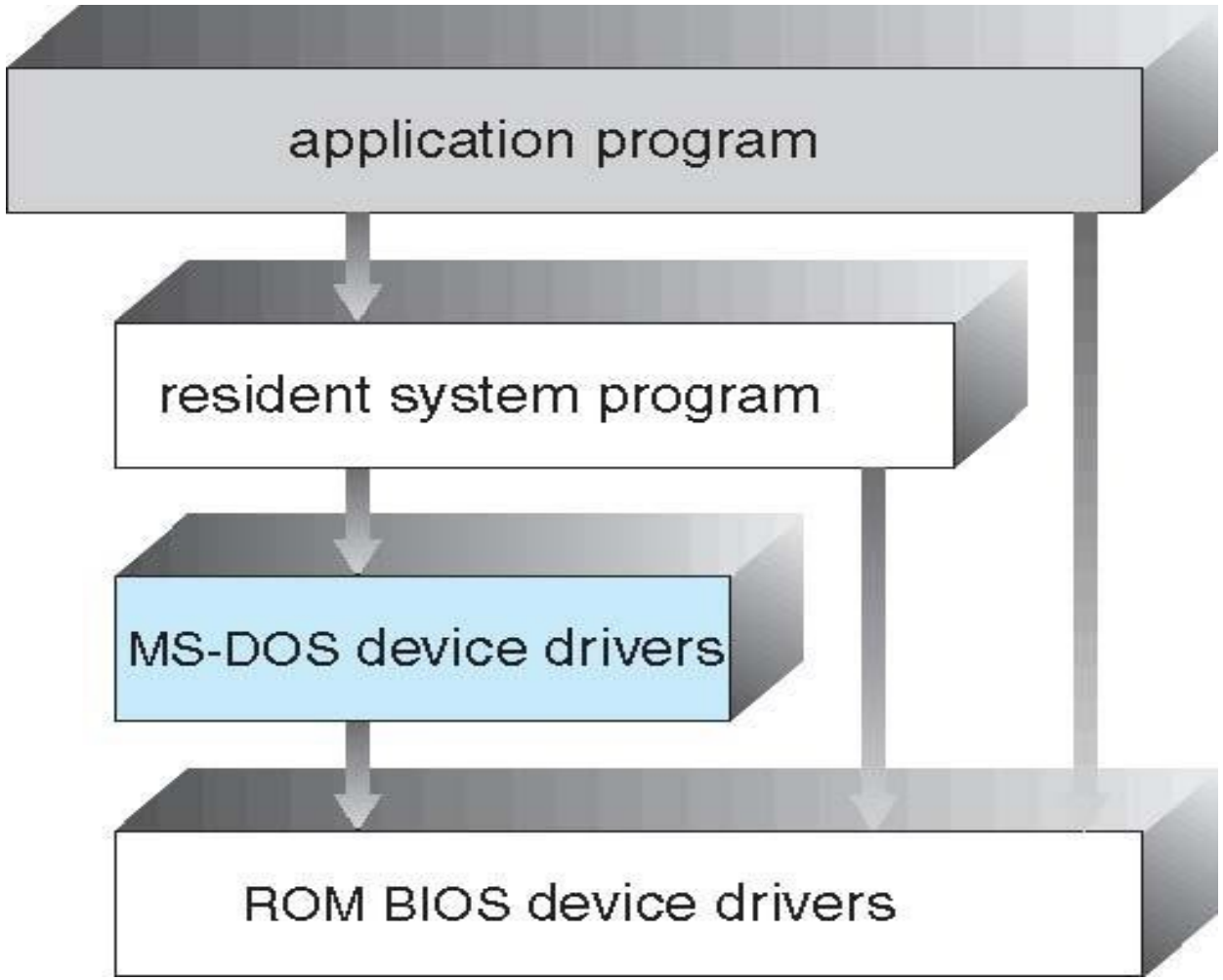
API – System Call – OS Relationship



Operating System Structure
Simple Structure

- MS-DOS – written to provide the most functionality in the least space
  - Not divided into modules
  - This kernel is hard to extend.
  - If a user wants to add a new service then the entire operating system has to be modified.
  - No dual-mode and no hardware protection
  - Leave vulnerable to errant or malicious programs
  - Crashes entire program when user program fails

MS-DOS Layer Structure



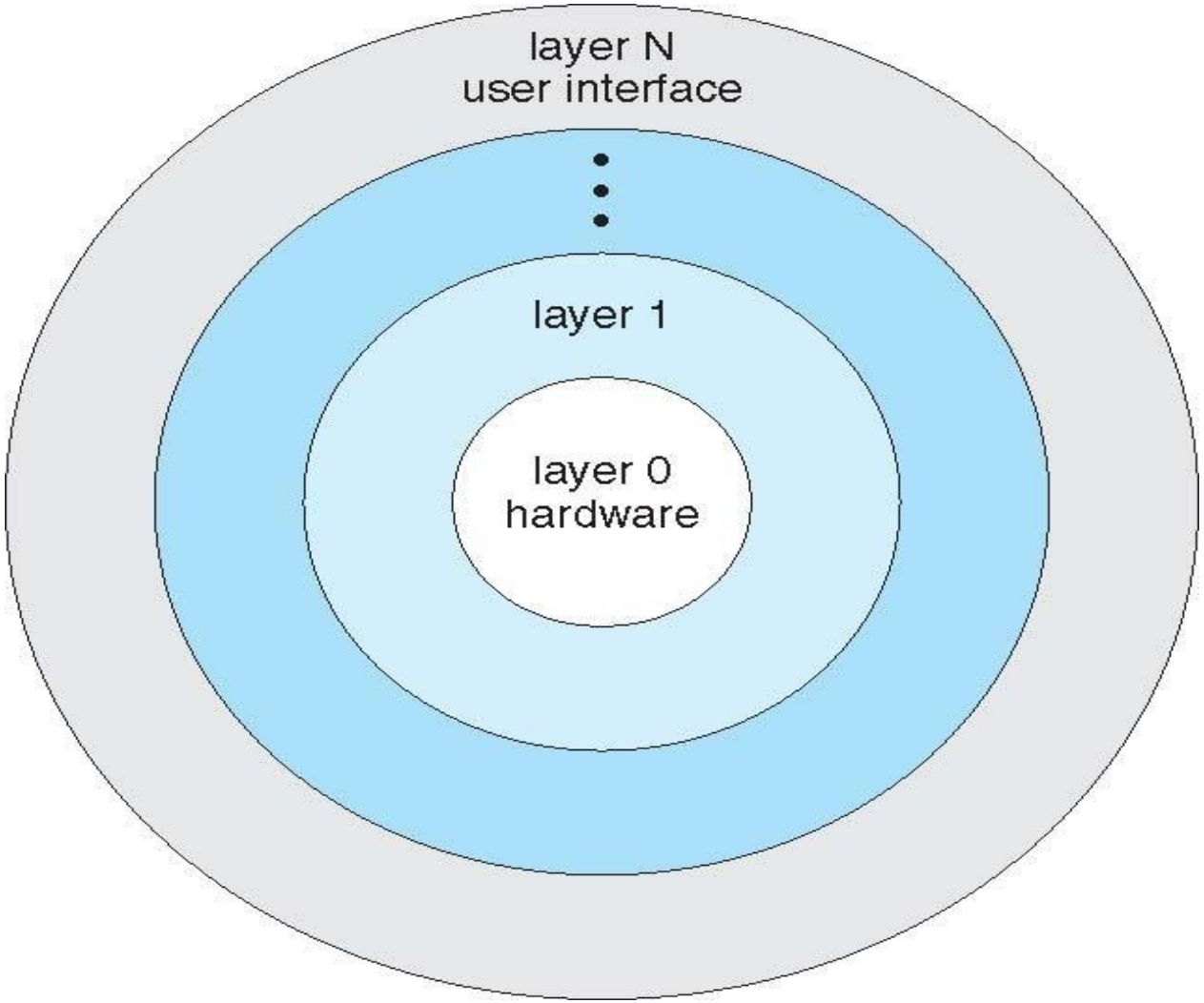
Layered Approach

- OS can be broken into smaller and more appropriate pieces creating modular operating system
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers
- More freedom in changing inner workings of the system

Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers

Layered Operating System



Layered Approach

- Advantages:
  - Simplicity of construction and debugging
  - Design and implementation of the system is simplified
    - Each layer is implemented only those operation provided by its lower layer
- Disadvantages:
  - Major difficulty involves appropriately defining each layer
  - Less efficient than others
    - An I/O request of user program adds overhead at each layer

Microkernel System Structure

- A microkernel-based system puts only the bare minimum system components in the kernel and runs the rest of them as user mode processes.
- Moves all non-essential components as much from the kernel into "*user*" space
- Implementing as system and user level
- Communication takes place between user modules using message passing

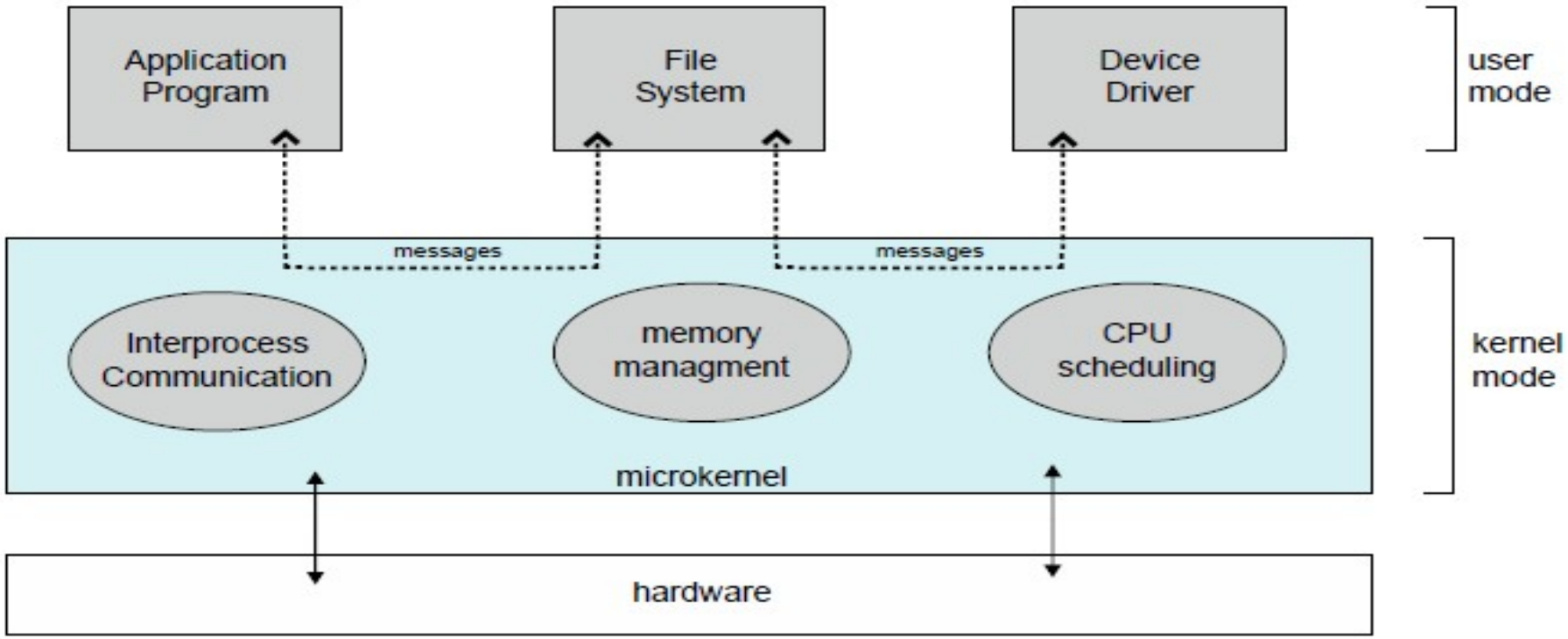
Microkernel System Structure

- new protocol stacks, file systems, device drivers and other low-level systems located in the monolithic kernel which results to a lot of work and careful code management.

- operating system functions, such as device drivers, protocol stacks and file systems, are typically removed from the microkernel itself and are instead run in user space.

- Windows NT became a hybrid kernel is speed
- Mac OS X kernel (**Darwin**) partly based on Mach

Microkernel System Structure



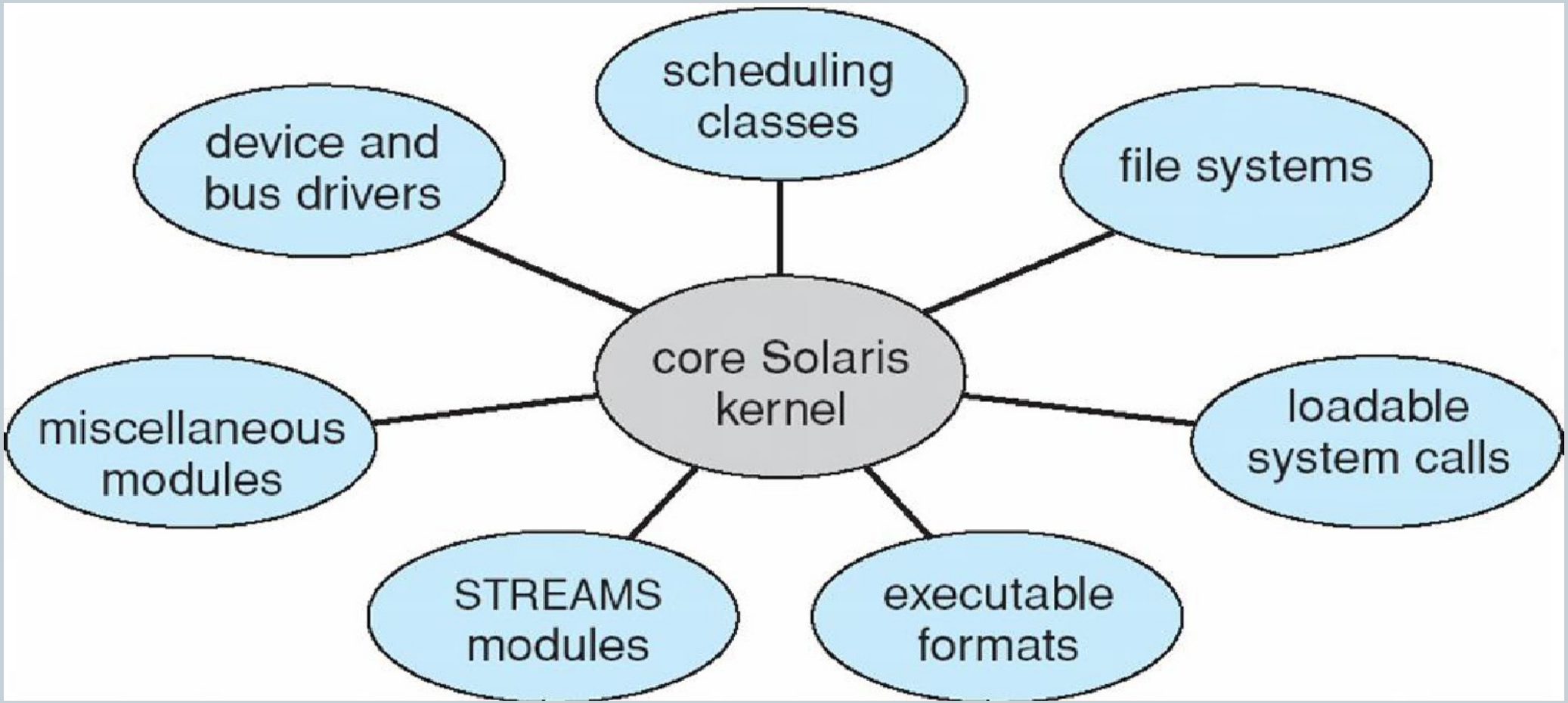
Microkernel System Structure

- Benefits:
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode)
  - More secure
- Demerits:
  - Performance overhead of user space to kernel space communication

Loadable Modules Architecture

- The best current methodology for OS design is loadable kernel modules where the kernel has a set of core components and links in additional services via modules either boot time or during run time.
- Kernel is provided to design core services
- And other services dynamically is adding new features directly to the kernel (required kernel compilation every time)
- Example: Solaris, Linux and Mac OS X.

Loadable Modules Architecture



Loadable Modules Architecture

- Advantages
  - More flexible than a layered approach as one module can call another module.
  - More efficient than microkernel because modules do not need to invoke message passing to communicate
- Disadvantages
  - the fragmentation penalty is a major disadvantage of loadable modules in the kernel. This means that every time a new kernel module code is inserted, the kernel becomes fragmented.[TLB (Transaction Lookaside Buffer)]

Hybrid Architecture

- Most modern operating systems are actually not one pure model
- Hybrid combines multiple approaches to address performance, security, usability needs
- Linux and Solaris kernels in kernel address space, so monolithic(Simple), plus modular for dynamic loading of functionality
- Windows mostly monolithic, plus microkernel for different subsystem ***personalities***

Virtual Machines

- A **Virtual Machine (VM)** is basically a **software-based computer** that runs inside your physical computer (host machine) just like a real computer. It behaves like a separate system with its own **OS, CPU, memory, and storage**, even though it's actually running on another machine.

Components of a VM:

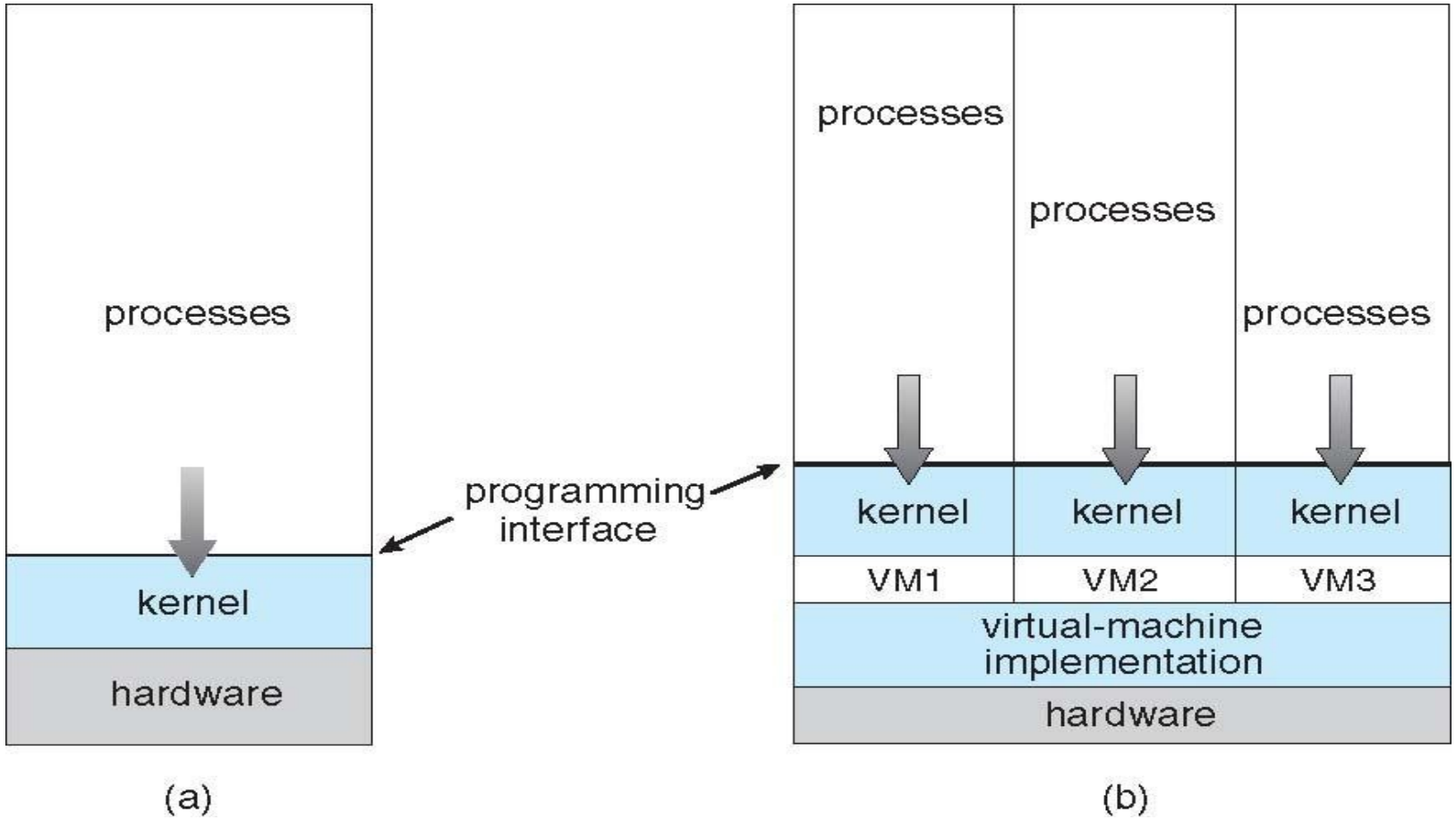
Guest OS → The operating system running inside the VM (e.g., Linux on Windows)

Virtual CPU, RAM, Storage → Allocated by the hypervisor

Virtual hardware → Network card, hard disk, display adapter

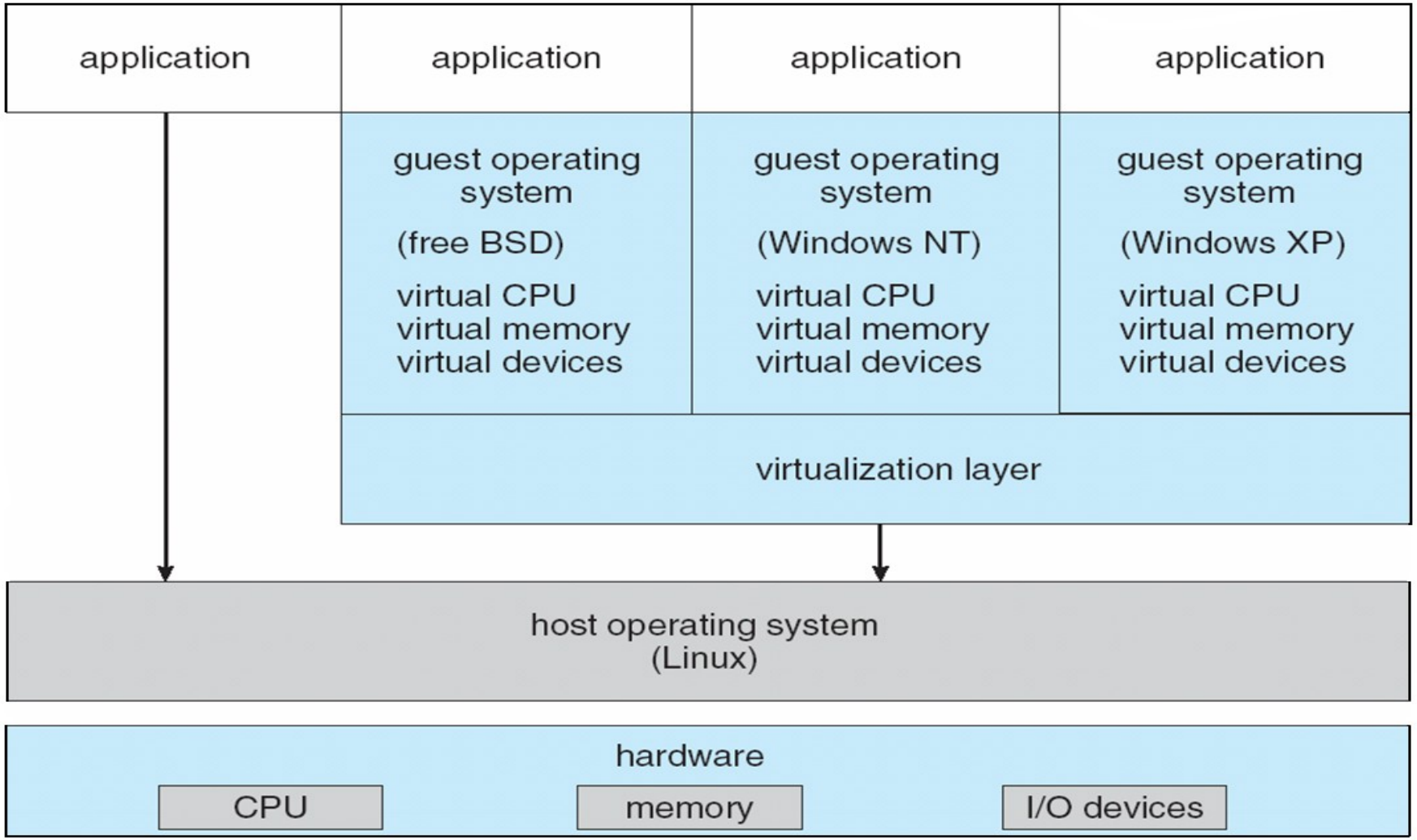
- The operating system **host** creates the illusion that a process has its own processor and (virtual memory).

Virtual Machines (Cont.)



(a) Nonvirtual machine (b) virtual machine

VMware Architecture



End of Chapter 2